

TP 4 : algorithmes de tris

ZFAI

Les questions marquées d'une $\star$ sont des questions théoriques. Elles peuvent être abordées plus tard, après le TP.

Exercice 1. Écrire une fonction Python `recherche(T,x)` qui est une traduction de l'algorithme de recherche dichotomique présenté dans le cours.

Exercice 2. Écrire une fonction `tri_comptage(L)` prenant en argument une liste L dont les valeurs sont comprises entre 0 et 50 et qui renvoie la liste triée dans l'ordre croissante correspondante.

Exercice 3. On donne le pseudo-code de l'algorithme de tri par sélection :

```
tri_selection(L)
/* L est une liste d'éléments à trier                                     */
n = longueur de L
pour i allant de 0 à n-1 inclus faire
    indice = i
    pour j allant de i + 1 à n-1 inclus faire
        si L[j] < L[indice] alors
            indice = j
        fin
    fin
    échanger L[i] et L[indice]
fin
```

1. À quoi correspond la variable `indice` à l'issue de la deuxième boucle ?
2. Écrire en Python le pseudo-code correspondant au tri par sélection. Pour échanger le contenu de deux variables en Python, on peut écrire $a, b = b, a$.

Exercice 4. 1. ($\star$) Montrer par récurrence qu'après k insertions, les k premiers éléments de la liste sont triés dans l'ordre croissant.

2. Écrire une fonction Python `tri_insertion(L)` prenant en argument une liste L et qui trie la liste par un algorithme de tri par insertion.

Exercice 5. Pour fusionner deux listes qui sont déjà triées, on peut utiliser l'algorithme récursif suivant :

```
fusion(L,M)
/* L, M sont deux listes déjà triées                                     */
si L ou M est la liste vide alors
    | renvoyer L+M
fin
si L[0] < M[0] alors
    | renvoyer [L[0]]+(fusion (L[1:],M))
fin
sinon
    | renvoyer [M[0]]+(fusion (L,M[1:]))
fin
```

1. Écrire une fonction `fusion(L,M)` prenant en arguments deux listes qui sont triées et qui renvoie la liste triée obtenue en fusionnant les deux listes. Par exemple, `fusion([1,3,5,6,9],[2,3,4,5,5])` renvoie `[1,2,3,3,4,5,5,6,9]`.
2. En déduire une fonction `tri_fusion(L)` qui renvoie la liste obtenue en triant L à l'aide d'un algorithme de tri fusion.
3. ($\star$) Justifier la correction de l'algorithme de `fusion`, c'est-à-dire que l'algorithme renvoie bien une liste triée à la fin de celui-ci. Pour cela, on peut démontrer la propriété suivante par récurrence sur la somme des deux longueurs : si $\text{len}(L) + \text{len}(M) \leq n$ et que L et M sont deux listes triées, alors la fusion des deux est triée.

Exercice 6. On propose une manière d'écrire un algorithme de tri rapide : il est récursif, fait intervenir une boucle et manipule la liste en place.

1. On considère la liste $L = [3, 5, 6, 1, 6, 7, 22, 1, 5]$. Appliquer l'algorithme donné avec `debut=0` et `fin=8` avant les appels récursifs. On décrira l'évolution de la liste L . Pour aider à la lisibilité, on pourra mettre en évidence les positions correspondant à `plus_petit` et `plus_grand`. Ainsi, on commence par

$[3, \underline{5}, 6, 1, 6, 7, 22, 1, \overline{5}]$

2. Justifier qu'à l'issue de la la boucle tant que et le dernier échange, le pivot est bien placé dans la liste.
3. Écrire en Python le pseudo-code donné. Comment trier complètement la liste ?
4. (★) Montrer qu'il existe des configurations où le tri rapide effectue un nombre de comparaisons de l'ordre de n^2 , où n est la taille de la liste.

```

tri_rapide_aux(L,debut,fin)
/* L est une liste, debut et fin sont des entiers correspondent à des indices de L. On cherche
   à trier la liste depuis debut à fin */
si debut < fin alors
    pivot = L[debut]
    plus_petit = debut + 1
    /* indice pour placer une valeur plus petite que le pivot */
    plus_grand = fin
    tant que plus_petit < plus_grand faire
        si L[plus_petit] > pivot alors
            échanger L[plus_petit] et L[plus_grand]
            plus_grand = plus_grand - 1
        fin
        sinon
            plus_petit = plus_petit + 1
        fin
    fin
    si pivot < L[plus_petit] alors
        échanger L[debut] et L[plus_petit-1]
        fin1 = plus_petit - 2
    fin
    sinon
        échanger L[debut] et L[plus_petit]
        fin1 = plus_petit - 1
    fin
    tri_rapide_aux (debut, fin1)
    tri_rapide_aux (fin1+2, fin)
fin

```

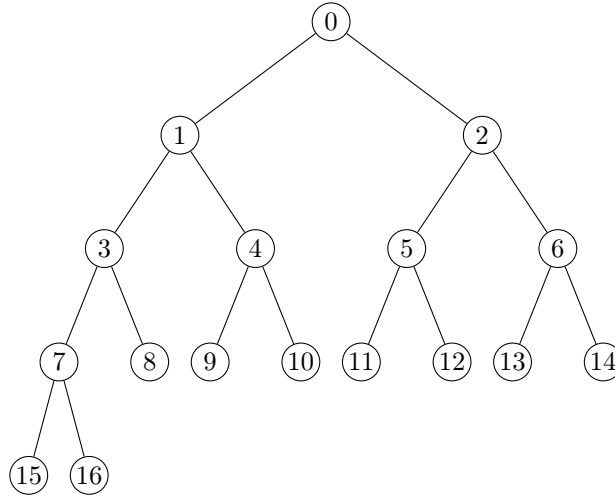
Pour les plus rapides : le tri par tas

On s'intéresse à un algorithme de tri basé sur une structure appelée tas. Il s'agit d'une liste où les éléments sont organisés d'une certaine façon.

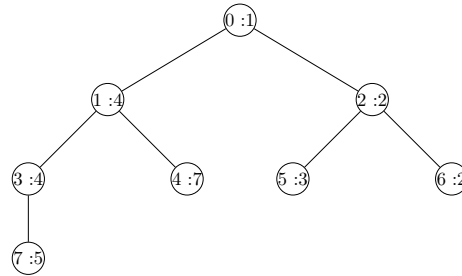
Étant donnée une liste de réels L de longueur n , on dit que L est un **tas** si la propriété suivante est vérifiée :

$$\forall i \in \{1, 2, \dots, n-1\}, L[i] \geq L[(i-1)//2]$$

Pour visualiser une structure de tas, on peut représenter la liste correspondante à l'aide d'un arbre :



les entiers écrits ici étant les indices des éléments de la liste. Représentons le tas suivant : $L = [1, 4, 2, 4, 7, 3, 2, 5]$.



dans cette représentation, dans un nœud, on a $i : L[i]$. On rappelle qu'en Python $L[i], L[j] = L[j], L[i]$ permet d'échanger les éléments d'indice i et j de L .

- Préciser pour chacune des listes suivantes si on a un tas ou non :
 (a) $L_1 = [2, 3, 4, 2, 5, 6]$ (b) $L_2 = [4, 1, 6, 5, 3]$ (c) $L_3 = [1, 2, 2, 5, 6, 4, 3]$ (d) $L_4 = [0, 0, 1, 1, 2, 4, 6]$.
- Écrire une fonction `est_tas(L)` prenant en argument une liste de nombres réels et qui renvoie `True` si L représente un tas et `False` sinon.
- Donner un ordre de grandeur du nombre maximum de comparaisons de la fonction `est_tas(L)` en fonction de la taille de la liste L .
- Écrire une fonction `diminue(L, i, a)` prenant en arguments une liste L représentant un tas, un entier i correspondant à un indice de L , un nombre réel a et qui remplace $L[i]$ par a dans le cas où $a \leq L[i]$ puis effectue des échanges pour que L garde une structure de tas. La fonction renvoie `True` si l'affectation a été effectuée et `False` sinon.
- Écrire une fonction `augmente(L, i, a)` prenant en arguments une liste L représentant un tas, un entier i correspondant à un indice de L , un nombre réel a et qui remplace $L[i]$ par a dans le cas où $a \geq L[i]$ puis effectue des échanges pour que L garde une structure de tas. La fonction renvoie `True` si l'affectation a été effectuée et `False` sinon.
- Écrire une fonction `ajout(L, a)` prenant en argument une liste L représentant un tas, un nombre réel a et ajoute cet élément a et qui effectue des échanges pour que L garde une structure de tas.
- Écrire une fonction `extraire_min(L)` prenant en argument une liste non vide représentant un tas et qui renvoie le minimum de L . La fonction effectue également la modification suivante dans L : elle remplace le minimum par la dernière valeur de L , supprime le dernier élément de L et effectue des échanges pour garder la structure de tas.
- Donner un ordre de grandeur de la complexité en nombre de comparaisons dans le pire cas de la fonction `ajout(L, a)` en fonction de la taille L .
- Écrire une fonction `construction_tas(L)` prenant en argument une liste L et qui renvoie le tas construit obtenu en initialisant une liste T vide et qui effectue des ajouts successifs des éléments de L dans T tout en gardant une structure de tas pour T .
- Donner un ordre de grandeur de la complexité en nombre de comparaisons dans le pire cas de la fonction `extraire_min(L)` en fonction de la taille L .
- Pour trier une liste, on peut procéder ainsi :
 — construire le tas associé à cette liste L ,

- extraire successivement les minimums jusqu'à ce que la liste soit vide.
- (a) Écrire une fonction `tri_par_tas(L)` qui effectue le tri décrit précédemment.
- (b) Donner un ordre de grandeur du nombre de comparaisons dans le pire cas lors du calcul de `tri_par_tas(L)` en fonction de la taille de L .